

Dockerización

Chatbot DianSistemas

Guía completa para desplegar el chatbot usando Docker y Docker Compose.

Tabla de Contenidos

1. Arquitectura de Contenedores
2. Inicio Rápido
3. Servicios en docker-compose.yml
 - Servicio setup (One-shot bootstrap)
 - Servicio web (Aplicación Django)
 - Servicios LLM: ollama y ollama-init
4. Configuración y Persistencia
 - Variables de entorno (.env)
 - Volúmenes persistentes
5. Operación y Mantenimiento
 - Comandos de gestión operativa
 - Reconstrucción y forzado de setup
6. Flujo de Primer Arranque y Monitorización
7. Consideraciones para Producción
 - Configuración de Nginx como Reverse Proxy
 - Dimensionamiento de recursos recomendado
 - Diagnóstico y Troubleshooting

1. Arquitectura de Contenedores

El entorno de despliegue se orquesta mediante Docker Compose, desacoplando los servicios de aplicación, inferencia de modelos LLM, procesamiento NLP y persistencia de datos en contenedores especializados:

Servicio	Imagen / Base	Rol y Responsabilidad	Detalles Técnicos
setup	Dockerfile (setup.sh)	Bootstrap inicial e inicialización	Ejecuta migraciones, carga de datos, compilación NLP y creación de modelos Ollama.
web	Dockerfile (Django)	Servidor de aplicación web	Gunicorn (2 workers, 4 threads, timeout 300s, puerto 8000).
ollama	ollama/ollama:latest	Servidor de inferencia LLM	REST API, healthcheck integrado, soporte opcional para aceleración GPU.
ollama-init	ollama/ollama:latest	Descarga de modelos base	Ejecuta el pull de llama3.2 una sola vez y finaliza.
db	mysql:8.4 (opcional)	Base de datos relacional	Perfil alternativo a SQLite para entornos corporativos.

2. Inicio Rápido

Pasos para clonar, configurar y desplegar el entorno en producción o desarrollo local:

```
# 1. Clonar el repositorio y acceder
git clone https://github.com/josecursoprogramacion-coder/Chatbot.git
cd Chatbot

# 2. Configurar variables de entorno
cp .env.example .env
# Editar .env con los valores correspondientes

# 3. Levantar la pila de contenedores en segundo plano
docker compose up -d

# 4. Monitorizar el proceso de inicialización (bootstrap)
docker compose logs -f setup
```

```
# 5. Acceder una vez finalizado el setup (Exit 0):  
# Chat: http://localhost:8000/chat/  
# Admin: http://localhost:8000/admin/
```

3. Servicios en docker-compose.yml

Servicio setup (One-shot bootstrap)

- Entrypoint: ./setup.sh
- Cadena de ejecución: migraciones Django → carga de datos iniciales → generación .spacy → entrenamiento spaCy → generación de Modelfiles → compilación de modelos en Ollama.
- Idempotencia: comprueba el marcador /data/.data_loaded y la presencia de modelos preexistentes.
- Dependencias: aguarda a que ollama esté saludable (healthy), ollama-init completado y db lista.

Servicio web (Aplicación Django)

```
Comando: gunicorn config.wsgi:application --bind 0.0.0.0:8000 --  
worker-class gthread --workers 2 --threads 4 --timeout 300
```

- Mapeo de puertos: 8000:8000.
- Dependencia: ejecución exitosa previa del contenedor setup (Exit 0).
- Variables clave: RUN_COLLECTSTATIC=1 para empaquetado de estáticos.

Servicios LLM: ollama y ollama-init

- ollama: Servidor base con API REST, volumen persistente ollama_data y healthcheck mediante 'ollama list' cada 10s.
- ollama-init: Tarea efímera que ejecuta 'ollama pull llama3.2' al arrancar.

4. Configuración y Persistencia

Variables de entorno (.env)

```
# Configuración Django  
SECRET_KEY=tu-clave-secreta  
DEBUG=False  
ALLOWED_HOSTS=localhost,127.0.0.1,tudominio.com  
  
# Conexión con Ollama en red Docker  
OLLAMA_HOST=http://ollama:11434  
  
# Base de datos SQLite por defecto  
SQLITE_PATH=/data/db.sqlite3  
  
# Configuración MySQL (solo activo con perfil mysql)  
DB_NAME=chatbot
```

```

DB_USER=chatbot
DB_PASSWORD=tu_password_seguro
DB_ROOT_PASSWORD=root_password_seguro

# Notificaciones por Email
EMAIL_HOST=smtp.tu_proveedor.com
EMAIL_PORT=587
EMAIL_USE_TLS=True
EMAIL_HOST_USER=tu_email
EMAIL_HOST_PASSWORD=tu_password
RESUMEN_EMAIL_DESTINATARIOS=admin@diansistemas.com

# Seguridad de integración web (iframe)
DOMINIOS_PERMITIDOS=https://tudominio.com, http://localhost:8000

```

Volúmenes persistentes

Volumen Docker	Punto de Montaje	Contenido y Propósito
app_data	/data	Base de datos SQLite (db.sqlite3) y marker .data_loaded.
spacy_model	/app/entrenamiento/ spacy/modelo	Modelos compilados y evaluados de spaCy.
ollama_data	/root/.ollama	Pesos binarios de modelos descargados (llama3.2).
mysql_data	/var/lib/mysql	Persistencia de base de datos MySQL (perfil mysql).

5. Operación y Mantenimiento

Comandos de gestión operativa

```

# Ver estado de los contenedores
docker compose ps

# Monitorizar logs en tiempo real
docker compose logs -f web
docker compose logs -f setup
docker compose logs -f ollama

# Detener la infraestructura
docker compose down

# Detener y eliminar volúmenes (¡Acción destructiva!)
docker compose down -v

```

Reconstrucción y forzado de setup

```
# Rebuild completo sin caché
docker compose build --no-cache && docker compose up -d

# Forzar recarga de datos en base de datos
docker compose run --rm -e FORCE_RELOAD=1 setup

# Forzar reentrenamiento completo de spaCy
docker compose run --rm -e FORCE_TRAIN=1 setup

# Forzar recarga de datos y reentrenamiento simultáneos
docker compose run --rm -e FORCE_RELOAD=1 -e FORCE_TRAIN=1 setup
```

6. Flujo de Primer Arranque y Monitorización

Durante el primer despliegue, el orden de inicialización asegura la integridad de los datos. El tiempo estimado del primer arranque se sitúa entre 5 y 15 minutos debido al entrenamiento del pipeline spaCy.

Paso	Servicio	Acción Ejecutada	Condición de Éxito
1	ollama	Arranque de motor LLM	Healthcheck OK (puerto 11434 responde)
2	ollama-init	Descarga de modelo llama3.2	Descarga finalizada (Exit 0)
3	setup	Migraciones y carga de datos	Tablas y registros creados
4	setup	Generación y entrenamiento spaCy	Artefacto model-best generado
5	setup	Generación de Modelfiles Ollama	Modelos chatbot registrados
6	web	Inicio del servidor Gunicorn	Servidor escuchando en :8000

7. Consideraciones para Producción

Configuración de Nginx como Reverse Proxy

```
server {
    listen 80;
    server_name tudominio.com;

    location / {
        proxy_pass http://web:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

```

    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;

    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";

    proxy_read_timeout 300s;
    proxy_send_timeout 300s;
}
}

```

Dimensionamiento de recursos recomendado

Componente	CPU Recomendada	Memoria RAM	Aceleración Opcional
web (gunicorn 2w/4t)	2 vCPU	2 GB	N/A
ollama (llama3.2)	2 vCPU	4 - 8 GB	NVIDIA CUDA (deploy block)
setup (entrenamiento)	2 vCPU	4 GB	N/A

Diagnóstico y Troubleshooting

Problema	Diagnóstico	Solución
Setup no termina	docker compose logs setup	Comprobar errores en migraciones o pipeline NLP
Setup finaliza con Exit 1	docker compose logs setup	Corregir causa y reiniciar con docker compose restart setup
Web no inicia	docker compose logs web	Verificar que el setup terminó con Exit 0
Ollama unhealthy	docker compose logs ollama	Ejecutar docker compose restart ollama
Puerto 8000 ocupado	docker compose ps	Modificar mapeo en docker-compose.yml (ej. '8080:8000')
Modelo spaCy no carga	docker compose logs web	Revisar SPACY_MODEL_PATH en el archivo .env